UNIVERSITY
OF SUSSEX

Masters Thesis

Placeholder title

A thesis submitted in fulfilment of the requirements
for the degree of Master of Research in Advanced Artificial Intelligence

in the

University of Sussex
School of Engineering and Informatics

Author:
Henry Williams

Supervisor:
Jeff Mitchell

August 2025

Contents

Chapter 1

Introduction

Word Count: 1000

- What is malware?
- Why is detecting it important?
- How do we detect it?
- How does it evade detection?
- What are we doing?
- Ethical & Safety considerations
  - Ethics — maybe better malware detection could be used to guide development of adverserial malware?
  - Safety — outline ways to ensure malware samples are never to be executed

`./introduction.tex` does not exist

Chapter 2

Background

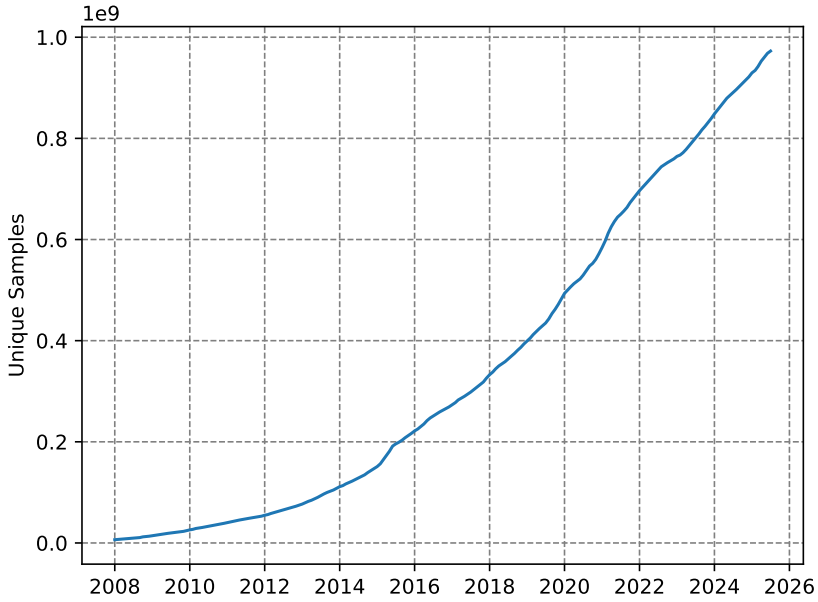


Figure 2.1: Observed growth in the number of unique forms of malware over time. Data taken from AV-TEST - The Independent IT-Security Institute, 2025

Malicious software, also known as malware, is defined as software that has an adverse impact on the expected operation of a computer system. This encompasses the provision of access by an unauthorised user to an infected host, illegitimate extraction and modification of information, or denying access to the host by its users. They are considered one of the most pervasive threats to both individuals and organisations in our current digital threat landscape (European Union Agency for Cybersecurity, 2024; Alenezi et al., 2020).

The threat posed by malware has been seen in cyberattacks such as the 2022 Viasat hack (Quiquet, 2023), and the 2017 WannaCry incident (National Audit Office (NAO), 2017; Akbanov et al., 2019). In these cases, sophisticated malware were used to disrupt critical infrastructure, including communication systems across Europe, and services used by the National Health Service.

In recent years, the sophistication (Alenezi et al., 2020; Baezner et al., 2017) and prevalance (Figure 2.1) of malware has increased significantly. As a result of this, and the potential damage caused by malicious software, there has been a great deal of interest into techniques for the reliable identification of malware (M. et al., 2023; Maniriho et al., 2024; Aboaoja et al., 2022). In this chapter, we provide a background on characteristics associated with malware, and review techniques proposed by security researchers to better manage this threat.

2.1 Malware

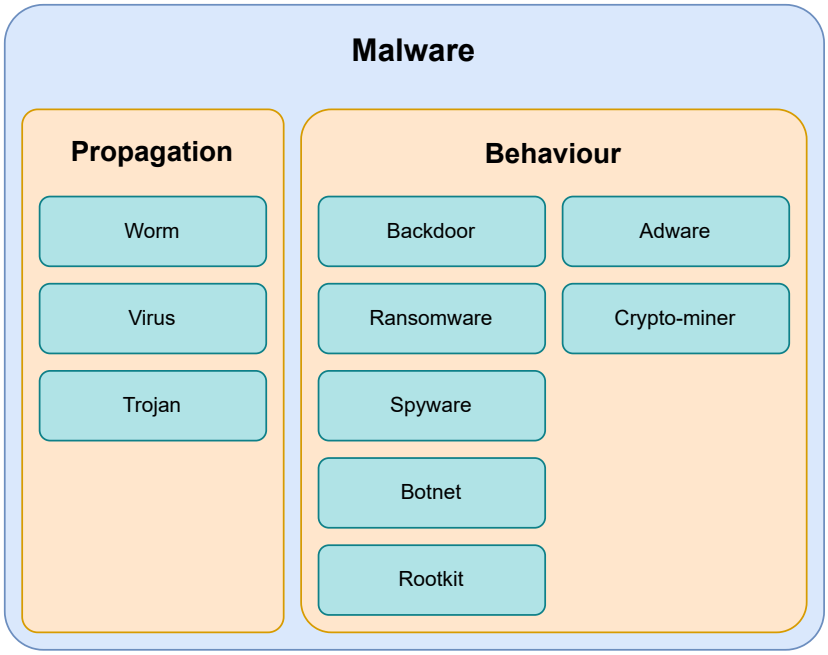


Figure 2.2: Categories of Malware, including both the means of propagation and behaviour on an infected host.

As outlined in the previous section, malware are a form of software that have a detrimental impact on the expected operation of an infected system. However, both this definition, and formal definitions, such as the one presented by (Kramer et al., 2010), fail to define what is meant by harmful. Therefore, we define harmful behaviour as actions that are in violation of the Confidentiality-Integrity-Availability (CIA) triad (Cochran, 2024). This is a principle used to guide security policies that outline the need for a system to be protected against unauthorised access (Confidentiality), modification (integrity), and reliable (availability) to be considered secure.

Malware can be categorised based on its behaviour, and its method of propagation. We present the following taxonomy based on the work of Maniriho et al., 2024 shown in Figure 2.2 that outlines the different kinds of malware seen by security researchers, and how they are spread. Note that categories within this taxonomy are not mutually exclusive, as they often belong to multiple categories at once.

- **Worm** — Malicious software that automatically propagates throughout a network.
- **Virus** — Programs that modify other programs to replicate themselves throughout an infected host.
- **Trojans** — Malware that disguises itself as a legitimate program to infect a host.
- **Backdoor** — Malware that provides covert remote access to infected systems.
- **Ransomware** — Malware that denies access to systems or information until a fee is paid.
- **Spyware** — Malware that spy on the user without their knowledge, often extracting information such as credentials, keyboard logs, and live screenshots of the user's activity.
- **Botnet** — Malware that integrate infected systems into a coordinated network of agents.
- **Rootkit** — Malware providing privileged access to an infected system.
- **Adware** — Malware that display advertisements on the host machine.
- **Crypto-miner** — Malware that hijack the user's hardware to mine cryptocurrencies.

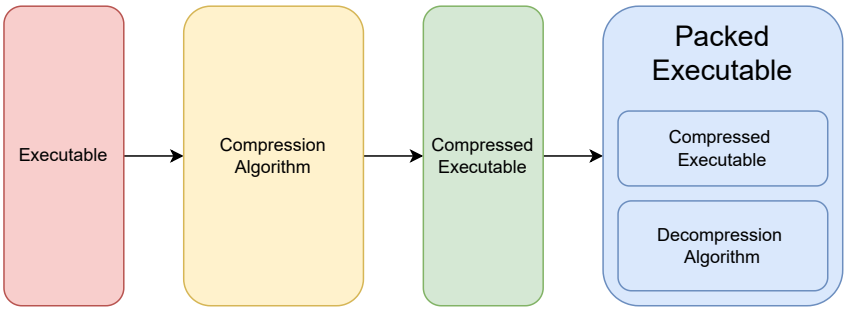


Figure 2.3: How binary packing can be used to reduce the size of an executable program, and obscure its behaviour.

2.1.1 Evasion Techniques

The earliest forms of malware detection used a combination of unique signatures associated with malicious programs, and a set of heuristics to identify malicious programs (A. Saeed et al., 2013). In response to this, developers of malicious software began using evasion techniques that aim to decieve these systems. This was done in order to maximise the amount of potential damage the software is able to do to an infected host before it is identified and removed. These evasion techniques include obfuscation, environmental awareness, and living-off-the-land. In the following section, we discuss these techniques, and why they are effective at defeating existing malware detection.

Obfuscation

Obfuscation is defined as the act of making something more challenging to understand. In the context of malware, it relates to techniques that alter the syntactic structure of a program, while maintaining its original behavioural characteristics. In this section, we explore different forms of obfuscation, and their effectiveness in evading detection.

Register re-assignment is the process of randomly reallocating registers used by the program (Balakrishnan et al., 2005). Registers are small areas of memory within the CPU that store information used during the program's execution. By changing which registers are used to store information, the physical structure of the program is changed, while maintaining its behavior. This defeats simple malware detection solutions that use the unique cryptographic hashes of a program as its signature.

Dead code insertion is simple form of obfuscation that inserts instructions which have no impact on the state of the program's execution. These instructions include the no-operation (`nop`), swapping the value stored in a register with itself, and adding zero to a register to name a few. This again modifies the physical structure of a program, without impacting its behaviour. This is not limited to singular instructions, and can include the insertion of functions which are never executed which can make the program more challenging to identify. However, dead code is relatively easy to identify and remove.

Code transposition modifies the order of code the program (Christodorescu et al., 2004) to obscure its signatures by moving small units of code known as basic blocks. These basic blocks do not contain jumps, such as loops or conditional statements. To ensure the program behaves as expected, all references to the relocated basic blocks are updated with its new location. This evades detection in a similar way to the obfuscation techniques discussed previously, by creating unique representations of the same program that give produce different signatures not recognised by the analysis software.

Despite the ability of these techniques to trick early anti-malware approaches, more sophisticated approaches are more resiliant to these forms of obfuscation. These approaches use an intermediate representation of the program, that normalise the structure of a program resulting in a representation that is indentical to the un-obfuscated representation. To evade these solutions, more sophisticated forms of obfuscation are needed. One such example of this is known as binary packing, where the malicious portion of code is encoded and stored as data within another program that decoded and executes the payload. Encoding can take many forms, such as encrpytion or compression, but they typically make it challenging to retrieve the decoded payload without running the program. However, the portion of the packed binary responsible for decoding and execution will often remain constant, meaning it can be used for identification. Some malware developers employ the aforementioned obfuscation techniques to obscure the decoder and avoid detection. This is known as Oligomorphic and Polymorphic malware (You et al., 2010).

Environmental Awareness

As a result of the previously discussed obfuscation techniques, relying on the static structure of a program may be insufficient to reliably detect malware. It is for this reason that dynamic features are considered to be more robust indicators of malicious programs, as the behaviour of a program is invariant to obfuscation. These dynamic features are observed at execution time and can include sequences of system API calls, modifications to the filesystem and system configuration, and internet traffic. However, this requires running a potentially malicious program, which could have harmful effects on the host system. It is for this reason that execution of a potentially malicious program occurs within a sandbox, an environment where the harmful actions can occur with no implications on the security of the system. This sandbox is typically a virtual machine or a containerized application, that's representative of the software's intended target.

To obscure the behaviour of the program, the developers of these malicious programs will attempt to check if the program is running within a virtual machine. Common heuristics used by malicious programs include system hardware characteristics (CPU core count, RAM size, etc), the presence of shared libraries used to communicate between the sandbox and its host, and the number of processes running on the potential host. If the program determines that it is likely to be running within a sandbox, it will refuse to execute the malicious portion of the code, and exit early. This prevents the observer from gaining any information about its behavioural characteristics, preventing the creation of dynamic malicious signatures.

Living-off-the-land

In recent years, the sophistication of malware has increased dramatically. Living off the land refers to an evasion technique used by advanced malware that significantly reduces the ability of existing security solutions to detect the presence of malware on a host system.

Upon gaining access to a system, often through software vulnerabilities, or social engineering, the malicious software will make use of trusted programs already present to further compromise the host. For example, certain tools in windows machines can make changes to the system registry, which is a trusted location that isn't typically analysed by anti-malware software. It remains consistent between system restarts, and includes a list of programs that should always be executed upon turning on. This allows a threat actor to install malicious software within a trusted location, that is persistent between restarts, that is not saved to the host's disk. This is known as fileless malware.

As the malware does not exist on disk, it is extremely difficult to detect by conventional means. Furthermore, as they rely on the functionality of trusted system programs, which are highly likely to be present on all targets.

2.2 Related Work

Malware detection is a challenging problem, and many approaches have been proposed since the first forms of malicious software were seen in the 1980s. The earliest form of malware detection used a database containing the unique signatures of malicious programs. While similar signature based approaches are still in use today, there is much interest in advanced methods, that are robust to evasion techniques, and that able to identify novel forms of malware (M. et al., 2023; Merabet et al., 2019; Aboaoja et al., 2022; Idika, Mathur, 2007). These advanced techniques employ a wide variety of technologies, such as multiple sequence alignment of API call sequences (Ki et al., 2015), machine learning (Kamboj et al., 2023), and deep learning.

These techniques typically fall into one of two categories, based on how the features used to classify programs are extracted which are as follows.

- **Static** — Features extracted from the structure of a program
- **Dynamic** — Features extracted during the execution of the program

Static approaches use features extracted from the contents of an executable, and do not require executing the program. Common features used to identify malware include greyscale images of the executable (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023), printable strings (Tian et al., 2009; Singh et al., 2020; Schultz et al., 2001), and sequences of instructions (Kakisim et al., 2022; Wang et al., 2021; Darem et al., 2021).

In Schultz et al., 2001, the authors use static features such as printable ASCII strings to train various machine learning classifiers to detect malware. In their experiments, they found that the Naive Bayes algorithm was best suited to the problem, achieving 97% classification accuracy. This is one of the first papers that introduce the usage of machine learning in this domain, and the high accuracy of their experiment shows its potential over conventional

signature or heuristic methods. However, the dataset used in this work was small, and featured an unbalanced class distribution of 3265 malicious programs, and only 1001 benign which would likely have an adverse effect on the generalisability of their proposed models. Furthermore, malware often obfuscate strings that are decoded during execution, which would be sufficient to defeat their proposed approach.

Much of the literature relating to static malware detection use convolutional neural networks to distinguish between malicious and benign programs (Ni et al., 2018; Jain et al., 2020; Habibi et al., 2023). In Aslan et al., 2021, the authors represent an executable as a greyscale image, and train a convolutional neural network (CNN) to detect malware. While their proposed approach is able to effectively distinguish between executable classes, CNNs require constant input size. Therefore, resizing of the image representation is necessary. Malicious programs are often smaller than benign programs, meaning that in datasets where there is significant variation in size between classes, compression artifacts may be more prevalant in once class, leading to the model developing an insufficient solution. In Sharma et al., 2019, the authors also use a CNN to detect malware. Instead of using a 2-dimensional CNN however, the authors opt to use a 1-dimensional variant. This led to better performance than other CNN based approaches, as the vertical spatial dependence, which is largely irrelevant in malware detection, is removed. However, it still has the same requirement as the previously discussed approach.

Instruction sequences have also been explored as static features in malware detection. For example, in (Attaluri et al., 2009; Runwal et al., 2012), the authors utilise hidden markov models and achieve promising results. As these models consider what actions are being performed by the host system without having to execute the program, they offer a solution that we believe to be more resilient to obfuscation than other static approaches.

Extending upon the idea of utilising instruction sequences for malware detection, (Demırcı et al., 2022) outline an approach in which they propose the use of stacked bidirectional Long-Short Term Memory (BiLSTM) and pre-trained language models including BERT (Devlin et al., 2019), and GPT-2 (Radford et al., n.d.). In their investigation, the BiLSTM model was able to achieve 98% classification accuracy whereas the pre-trained transformer models achieved only 77%. Transformer models have been shown to be a powerful technique for sequence classification problems, and as such, this result is surprising. We believe this to be a result of the corpora used to pre-train the transformer models used in their investigation, which are comprised of mostly english texts. The understanding of these models would thus be limited to languages similar to english, and may struggle to understand the intricacies of assembly language, impacting their ability to reliably classify malware.

Previous research has also established the use of pre-trained language models in malware detection on the metadata of an executable. In Rahali et al., 2021, the authors fine-tune BERT on text gathered from the manifest file found in android applications. This file contains information pertaining to the activities, background tasks, event listeners, and permissions granted to an application. Their model was able to achieve 97% binary classification accuracy, in comparison to (Demırcı et al., 2022) which was only able to achieve 77% with the same model, trained on instruction sequences. We believe this discrepancy to be a result of the language found in these manifest files being more closely related to the pre-training corpora of the BERT model. This would enable their model to better understand the information found in the manifest, and its use in distinguishing between benign and malicious executables. Their approach however, is limited to environments where a file containing metadata are required, such as mobile operating systems, making it unsuitable for desktop malware detection.

The performance of static malware detection approaches vary, depending on the detection method used, the availability of data, and other factors, but it is well established that they are particularly vulnerable to obfuscation. In Bacci et al., 2018, the authors investigate the impact of obfuscation on the performance of static and dynamic malware detectors on android devices. They found that while dynamic methods are not significantly impacted by the presence of obfuscation, static techniques are more affected. However, by training static methods on obfuscated programs, it is possible to mitigate this problem to some extent.

Furthermore, dynamic methods generally tend to be more capable than static methods (Damodaran et al., 2017). This is somewhat to be expected, as behaviour is more indicative of malicious actions, especially when obfuscation is used to hide a program's intent. However, dynamic methods require a sandbox to execute a program for analysis, without causing damage to a host. This typically takes the form of a remote, virtual machine as running such a sandbox locally would have a significant computational cost. As a result of this, dynamic methods require both significant compute resources, and an internet connection, which is not guaranteed. It is for this reason that static methods, that can identify potentially malicious programs, without an internet connection is needed for enhanced protection from this threat.

In addition to the high resource requirements associated with dynamic feature extraction, sophisticated evasion techniques such as environmental awareness and living off the land provide malware developers with the capabilities to obscure the dynamic features. For example, by either preventing execution within virtual machines, or using trusted binaries already located on a host system. Furthermore, novel obfuscation techniques, such as the one

proposed by Li et al., 2023, is able to obscure which system API calls are made by a program. This is a widely used dynamic feature for detecting malware, and the proposed technique enables the malware to call these system APIs without triggering system hooks used to track which methods are called during a program's execution. Therefore, despite offering better performance than static methods, dynamic methods are also vulnerable to evasion techniques.

2.3 Datasets

Research into malware detection depends on the availability of large scale, high quality datasets containing both malicious and benign software. However, very few of these datasets exist, and those that do often provide features derived from the programs as opposed to the executables themselves. For example, the EMBER dataset (Anderson et al., 2018) contains over a million samples of static features derived from each executable, however the binaries themselves are not distributed. Other datasets, such as BODMAS (Yang et al., 2021), despite offering fewer samples than EMBER, provide access to the executables. However, due to copyright limitations, only malicious binaries are provided.

When curating these datasets, malicious samples are easy to download en-masse through websites such as VirusShare[1] and VirusExchange[2]. These websites store large collections of malware seen in-the-wild by security professionals. However, similar large scale collections of benign software are less available, likely due to copyright protections. We have seen a number of approaches for curation of benign software in the literature. For example, by downloading android applications from the Google play store (Karbab et al., 2018), scraping free software websites (Li et al., 2022), and extracting system executables. These approaches however are somewhat limited by the number of programs available on both free software websites, and the system itself. Furthermore, these executables are not guaranteed to be free of malware. Another promising approach for collecting benign windows software is the downloading of repositories used by windows package managers such as chocolatey and Winget as they provide many packages that have been verified as being free of malware. However, they often distribute installers rather than the executables themselves, meaning each package has to be installed before it can be used in the dataset.

2.4 Future Research Directions

Our analysis of the literature relating to this field, has found that there are a number of promising areas of investigation. Static approaches, while less robust to structural obfuscation, are less resource intensive and are able to run on a users machine without an internet connection. Dynamic features offer better performance than its static counterparts, yet are still vulnerable to advanced evasion techniques that may render them less effective than static. Furthermore, the requirements associated with dynamic feature extraction are a significant barrier to their deployment, and demonstrate the need for efficient and effective static methods for detecting malware.

Furthermore, we identify a gap in the literature relating to the use of modern natural language processing techniques such as pretrained transformer models in malware detection. While these models have been used in this context, they use the natural langauge features of these programs, such as the ascii strings, and program metadata. We will investigate the use of these technqiues when trained to develop an understanding of the program itself to see if these modern NLP techniques are suited to reasoning about executables for their use in detecting malware.

In addition to this, we will investigate if static methods learn to detect malware based on the presence of obfuscation, as it seems that the prevalance of obfuscated malware is significant in real world cybersecurity incidents. This will develop a better understanding of the issues associated with static malware detection.

[1] `https://virusshare.com`
[2] `https://virus.exchange/`

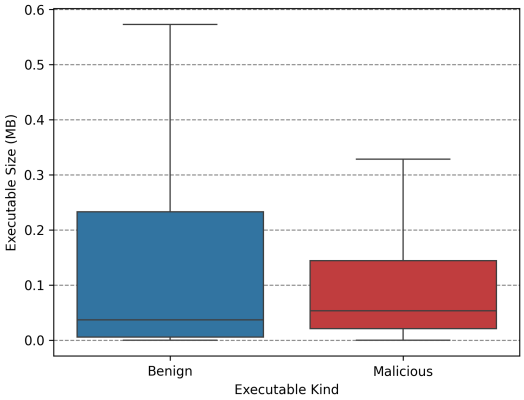
Chapter 3

Methodology

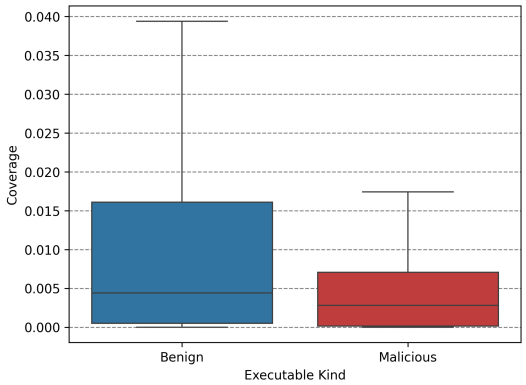
In our investigation, we develop a static malware detection method inspired by modern Natural Language Processing techniques, to establish if they are well suited to the identification of malware. We develop a model inspired by the architecture, and training procedures of BERT and other modern transformer based language models that is able to detect malicious software, without the significant overhead associated with dynamic approaches. This model is then compared against an image-based CNN approach, which is a common approach seen in the literature.

We also aim to establish a better understanding of the challenges faced by existing static approaches to malware detection, by testing both approaches on a dataset of unobfuscated executables.

3.1 Dataset



(a) Distribution of executable size across benign and malicious programs



(b) Distribution of code analysis coverage of benign and malicious programs

Class	Count
Malicious	5792
Benign	5323
Total	11115

(c) Distribution of classes with our dataset.

Figure 3.1: Description of our dataset, including the size of programs (a), the extent of obfuscation (b), and the total number of samples in each class (c).

Our dataset consists of 11115 windows PE32 executables (Figure 3.1c). Malicious files were gathered from VirusShare[1], which contains malware samples collected from real-world cybersecurity incidents with over 90 million samples. The

[1] `https://virusshare.com/about`

benign executables were collected from the personal devices of the authors, and were scanned using Microsoft defender to ensure no mislabelled samples were present.

Windows was selected as the target for this investigation due to the high availability of malicious software that attack this operating system. While malware that target other operating systems, such as MacOS and Linux, do exist, there are significantly fewer than those targeting windows. Therefore, each of our samples are stored in the Windows Portable Executable format. In addition to the logic and data of the program, this format also include various metadata that provide all the necessary information to the operating system to run the program. This includes external references, code size, and instruction set to name a few.

Our dataset demonstrates wide variation in characteristics between classes. For example, malicious programs are often much smaller than our benign executables (Figure 3.1a). Benign programs typically provide more functionality, such as Graphical User Interfaces and custom configuration options that will increase the size of the program. Malicious programs on the other hand, are much smaller, as typically do not require additional complexity that would increase the size of the program.

We believe that there is significant variation in the extent of obfuscation between classes. We use a static code analysis tool known as Rizin[2] to find the analysis coverage. This is the ratio of total code analysed to the total amount of code, and a lower value typically indicates a greater extent of obfuscation. In Figure 3.1b, find the distributions of analysis coverage between classes which shows that there is significantly more obfuscation in the malicious executables.

3.1.1 Obfuscation Experiment Dataset

Class	**Count**
Malicious	60
Benign	61
Total	121

Figure 3.2: Distribution of classes with our obfuscation experiment dataset.

To investigate the impact of obfuscation on malware detection, we have also curated a secondary dataset, consisting of unobfuscated executables. To achieve this, we use the work presented by (Rokon et al., 2020) in which they use machine learning to identify the github repositories of malware. From this list, we identify malware targeting windows, that include an executable, and without obfuscation. The executables comprise the malicious portion of this dataset, and the benign programs consist of Windows XP system binaries.

3.2 Metrics

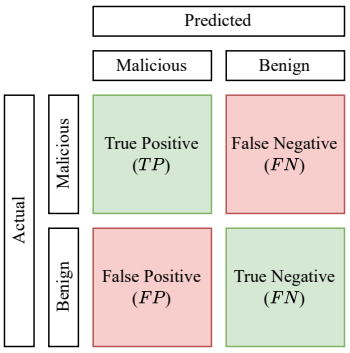


Figure 3.3: Confusion Matrix outlining the kinds of classification results in malware detection.

[2] `https://github.com/rizinorg/rizin`

In our experiments, we present malware detection as a binary classification problem, where our models are to predict one of two class labels, representing benign and malicious programs. To quantify the performance of the models we test in this investigation, we use Accuracy, F1-score, Precision, and Recall shown in Equation 3.1. These metrics are based on the different kinds of classification results of our models, as shown in Figure 3.3

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \\ \text{F1-Score} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned} \tag{3.1}$$

3.3 Baseline

Similarity	Malicious	Benign
Malicious	0.1428	0.0931
Benign	0.0931	0.1217

(a) Cross-class external function similarity

Similarity	Malicious	Benign
Malicious	0.2249	0.1606
Benign	0.1606	0.1643

(b) Cross-class DLL similarity

Figure 3.4: Similarity between classes based on external functions and DLLs

We firstly establish a baseline using the conventional machine learning algorithms and a list of external references associated with each program. These external references tell the program the location of libraries and functions provided by the operating system. For example, in order to interact with the host file system, a reference to the `KERNEL32.dll` will be present. Both the library, and the specific function within said library used by the program are present, and can be easily extracted with tools such as radare2[3]. References with a single occurence across our dataset are removed, and a binary feature vector for both libraries and functions are produced for each sample in our dataset. We test 3 machine learning algorithms for our baseline, namely support vector classifiers, logistic regression, and decision trees.

In addition to the baseline approaches, we also use the feature vectors to quantify the similarity of programs with our dataset. We use the cosine similarity function (Equation 3.2) to find the pairwise similarity of all samples. We then take the average similarity between classes which are presented in Figure 3.4. From this, we can assume that functions and libraries used by the malicious programs are more similar than the ones used by benign programs.

$$S(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{3.2}$$

3.4 RoBERTa for Opcode Sequence Classification

```
ret lea mov add add lea mov add add nop
```

Figure 3.5: An example sequence of opcodes

In recent years, transformer-based architectures for natural language processing have been shown to be highly effective across a wide variety of language based tasks. A significant advancement within the field was the development of the Bidirectional Encoded Representation from Transformers model, known as BERT (Devlin et al., 2019). This model uses an encoder-only architecture, based on the multi-head self-attention mechanism (Vaswani et al., 2023) in conjunction with a pretraining stage to develop a deep understanding of language. Pretraining consists of two tasks, masked language modelling where the model predicts specific tokens based on their surrounding context, and next sentence prediction. The pretrained model is then fine-tuned on a downstream problem such as sequence

[3] `https://github.com/radareorg/radare2`

classification, question answering, and general language understanding. At the time of publication, it was able to achieve state-of-the-art performance in General Language Understanding Evaluation (GLUE), Stanford Question Answering Dataset (SQuAD), and Situations With Adversarial Generations (SWAG) benchmarks and has been used extensively throughout the field. Fine-tuning of the pretrained model allows researchers to solve tasks with fewer training samples, short training periods, and less intensive compute requirements. RoBERTa extends upon the original BERT model by expanding the number of parameters, removing the next sentence prediction task from the pretraining stage, optimising its hyperparameters, and training on a larger corpus.

We develop a custom RoBERTa model, trained on opcode sequences for malware detection. Opcodes refer to the operation to be performed by the CPU, such as `add`, `cmp`, `mov`. These sequences are extracted from the disassembly of a program generated with radare2. The operands are then removed to reduce the vocabulary size of the model giving a collection of opcode sequences as shown in Figure 3.5 for each program within the dataset. These opcode sequences are tokenized with a word level tokenizer, where each token is a single word within the corpus as the vocabulary size is relatively low following the removal of operands.

We train three sequence classification models that predict the class label of a sequence of opcodes. The first uses a pretrained model, trained on masked language modelling of opcode sequences. We also train a model again using the same pretrained model, but with all frozen weights in the attention layers, meaning only the classification head is trained. This significantly reduces the time spent training, and the compute required to train the model, however it can lead to suboptimal performance. Finally, we train a model from scratch to see if pretraining is necessary in this context.

Our models share the architecture shown in Table 3.1 that was found to be the optimal configuration after an extensive hyperparameter sweep of 90 runs. Training was performed with a single Nvidia A40 GPU, and 16GB of ram with a batch size of 128 for a single epoch.

For each sequence of opcodes, with a fixed length the model predicts its class label, either malicious or benign. However, due to limited compute resources, we are unable to perform sequence classification over the entire sequence of opcodes for a single program. Therefore, to determine the class of a program, we need to reduce a collection of class likelihood values into a single pair or logits. We tested both mean reduction, where we take the average class likelihood for each over the sequence of logits, and majority choice where the most common result is selected. Of these methods, we found mean reduction to be most effective.

Parameter	**Value**
Sequence Length	32
Hidden Layer Size	512
Hidden Layers	7
Attention Heads	4
Intermediate Layer Size	2048
Activation Function	GELU
Hidden Dropout Rate	0.01
Attention Dropout Rate	0.001

Table 3.1: RoBERTa model parameters

3.5 CNN for Image Classification

In addition to the RoBERTa sequence classification model, we also develop an image based CNN approach. We use the EfficientNet-B0 architecture (Tan et al., 2020), due to its computational efficiency and high performance on image classification problems. The model is trained from scratch, and we modify the final layer of the model to output binary class likelihood values. Programs are converted to greyscale images using the procedure outlined in section A.1, and then converted to RGB before use by the model.

As previously established, the size of benign programs is typically much higher than malicious programs. As a result of this, benign images are more significantly resized than malicious images. Resizing an image will introduce artifacts to the image, such as anti-aliasing or blurring. Therefore, benign images may have more prevalent artifacts than malicious images which could be used by the model to learn a suboptimal solution. To prevent this from occurring, we test the same model on a subset of the dataset where the distribution of program size between classes are more similar. We use a genetic algorithm that minimises the Kullback-Leibler divergence D_{KL} between the

distributions of program size for each class of program by modelling it as a Knapsack problem (Khuri et al., 1994). We also encourage the algorithm to prefer solutions that have a high ratio of malicious to benign samples to ensure the class distribution is approximately equal.

We train both the baseline model, and the model trained on a curated dataset for 20 epochs, with the binary cross entropy loss function and a batch size of 32. Training was conducted using an Apple M2 Pro CPU, leveraging the metal performance shaders backend for hardware acceleration, with 16GB of RAM.

3.6 Obfuscation

Obfuscation is highly pervasive within malware seen in real world security incidents. As a result of this, we believe that static malware detection methods learn to identify obfuscation as opposed to other malicious characteristics. However, obfuscation is not exclusive to malicious programs, as legitimate software developers may use similar techniques to protect their intellectual property. Should this be the case, legitimate benign software that have been obfuscated may be misidentified as malicious, thereby degrading the effectiveness of these approaches in real world deployment.

We investigate this by observing how the models performance changes as we increase the presence of obfuscation within the unobfuscated dataset. We perform three trials for each model, that vary which kinds of program are to be obfuscated. For example, only malicious, benign, and all programs. At each step, we obfuscate a certain percentage of programs within the dataset and this percentage increases by 10% upon completion. We collect the logits for each sequence, which are used to calculate both the sequence level, and the file level performance. We also perform this experiment on the image classification model to see if these models are better suited to obfuscation resiliant malware detection.

Obfuscation is performed using a tool known as UPX, which is a binary packing tool. We chose this tool due to its extensive compatability with windows executables. Furthermore, a number of our malicious executables have been identified as using this tool for obfuscation, and it has also been used to compress and obfuscate benign files. Therefore, we feel that it is a suitable tool to replicate the forms of obfuscation used by both malicious and legitimate software developers seen in the wild without having to implement our own obfuscation tooling.

Chapter 4

Results

Word Count: 3000

- Baseline results
  - All hover around 90% accuracy
  - Functions are marginally better than DLLs
- Opcode model
  - Is pretraining useful?
  - How best to reduce sequence-level classification results to a file-level result?
- Image model
  - Is increased classification accuracy a result of the difference between the distribution between the size of malicious and benign programs?
- Obfuscation Experiment
  - Opcode Model
    - Baseline file-level classification accuracy on secondary dataset
    - When both classes of programs, or soley malicious programs are obfuscated, performance increases
    - When only benign programs are obfuscated, accuracy doesn't increase
    - Less pronounced in sequence level tasks
    - Certainty (D_{KL}) of classification increases as more programs are obfuscated
  - Image model
    - Effect seems to be more pronounced in filtered model
    - Increase in classification accuracy when only mal obfuscated
    - Decrease in c.a. when only ben obfuscated
    - Similar in base model but less pronounced
    - Confidence of base model increases when all and/or ben are obfuscated
    - Confidence when mal obfuscated decreases certainty
    - Filtered model less confident overall, little impact in obfuscation
  - TODO: Retrain both models on dataset with obfuscated benign train data & repeat previous experiments

Algorithm	Feature	Accuracy	F1 Score	Precision	Recall
Support Vector Classifier	DLL	0.89	0.89	0.89	0.89
	Function	0.92	0.92	0.92	0.92
Logistic Regression	DLL	0.87	0.87	0.87	0.87
	Function	0.91	0.91	0.92	0.91
Decision Tree	DLL	0.88	0.88	0.89	0.89
	Function	0.93	0.93	0.93	0.93

Table 4.1: Binary classification metrics of our baseline approaches.

4.1 Baseline

The results of our baseline approaches are presented in Table 4.1, and indicate the methods we tested are comparible in terms of their performance. The F1 score is approximately 0.9 across models trained on external functions used by programs, and is approximately 0.89 across models trained on DLL information. This suggests that external functions are a better indicator of a programs intent than the DLLs they are members of.

Our earlier analysis of import tables throughout our dataset support this result, as we were able to show that the average cosine similarity of external libraries (DLLs) used by malicious programs is greater than that of the external functions. This is also seen in the average cross-class pairwise similarity, and the average benign pairwise similarity, indicating that the DLLs used by a program are too general to be indicative of a programs nature.

Of all the methods tested, we found the decision tree classifier to be the best at identifying malicious software based on the usage of external functions. For external libraries, Support Vector Machines offered the best performance over others.

4.2 Sequence Classification

Model	Sequence Level				File Level			
	Accuracy	F1-Score	Precision	Recall	Accuracy	F1-Score	Precision	Recall
From Scratch	0.8756	0.8742	0.8798	0.8756	0.8976	0.8971	0.9095	0.8989
From Pretrained	0.9010	0.9002	0.9042	0.9010	0.9452	0.9452	0.9479	0.9458
From Pretrained (w/ Frozen Attention Weights)	0.8302	0.8265	0.8409	0.8302	0.8076	0.8037	0.8393	0.8098

Table 4.2: Performance of our RoBERTa model for opcode sequence classification and malware detection

The results for our NLP based approach to detecting malware are presented in Table 4.2. We find that on sequence level problems, where the model is tasked with identifying malicious opcode sequences, the pretrained model is comparible in terms of performance to our best baseline approaches (Table 4.1). When the attention layers are frozen however, performance is degraded by a significant margin. It is well established that an increase in model complexity is directly correlated with its performance (Kaplan et al., 2020). The model architecture (shown in Table 3.1) used in our investigation has approximately $2.3e+7$ parameters of which approximately 1% are used for classification[1]. Therefore, in the fine-tuned model, the complexity is significantly lower than the other models, meaning it is unable to reliably classify opcode sequences. Pretraining has a positive impact on the models sequence classification ability, as shown in the discepancy between the pretrained model, and the model trained from scratch.

However, these results only consider the sequence level accuracy. In order to identify malicious programs, we would need to aggregate each of these results into a single result for the file itself. To find this result, we take the mean of each class likelihood value for all sequence classification results. From this, we see the file level accuracy increase for both the pretrained model, and the model trained from scratch. However, the pretrained model with

[1] 23011330 in total, 263682 used in classification head

frozen attention weights actually decreases in terms of its performance, indicating the reduction has an adverse impact on its performance. We believe this to be caused by the fact that prior to its reduction, approximately 83% of the sequences will be classified correctly. In comparison to the other models which are able to more reliably classify sequences,

4.3 Image Classification

4.4 Obfuscation Experiment

4.5 Analysis

Chapter 5

Discussion

Word Count: 3000

- Are dynamic methods vulnerable to advanced obfuscation techniques?
  - It's possible to hide API calls from standard API call hooking techniques
  - Other redundant API calls may be made to obfuscate call sequences
  - Advanced malware (i.e. Rootkits) are able to stealthily perform malicious actions on the host
- Are opcode sequences suitable features for NLP based malware detection?
  - Full disassembly may offer provide better information for transformer architectures
  - Was not tested due to time/compute limitations
  - Byte-level modelling may work, but the presence of polymorphic malware and encrypted payloads may be challenging to model accurately
- Is executable packing (UPX) a good proxy for the obfuscation methods seen in the wild?
  - What other obfuscation techniques and solutions could work for this?
- Could diffusion modelling enable machine de-obfuscation?
  - Obfuscation is analgous to noise in a signal
  - Diffusion models are well suited to denoising of data
  - Might be able to de-obfuscate programs to generate large scale
- Data limitations
  - Lack of large-scale, benign executable datasets
  - High presence of obfuscation in real-world malware samples
  - Unobfuscated malware samples are not representative of malware seen in the wild as they are often "toy" projects, and lack advanced capabilities
  - Unobfuscated, advanced malware are hard to find
- Conclusion

Bibliography

A. Saeed, Imtithal, Ali Selamat, and Ali M. A. Abuagoub (Apr. 2013). "A Survey on Malware and Malware Detection Systems". In: *International Journal of Computer Applications* 67.16, pp. 25–31. ISSN: 09758887. DOI: 10.5120/11480-7108. (Visited on 07/29/2025).

Aboaoja, Faitouri A. et al. (Jan. 2022). "Malware Detection Issues, Challenges, and Future Directions: A Survey". In: *Applied Sciences* 12.17, p. 8482. ISSN: 2076-3417. DOI: 10.3390/app12178482. (Visited on 12/10/2024).

Akbanov, Maxat, Vassilios G Vassilakis, and Michael D Logothetis (2019). "WannaCry Ransomware: Analysis of Infection, Persistence, Recovery Prevention and Propagation Mechanisms". In: *Journal of Telecommunications and Information Technology* 1, pp. 113–124.

Alenezi, Mohammed et al. (Dec. 2020). "Evolution of Malware Threats and Techniques: A Review". In: *International Journal of Communication Networks and Information Security* 12, p. 326. DOI: 10.17762/ijcnis.v12i3.4723.

Anderson, Hyrum S. and Phil Roth (Apr. 2018). *EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models*. DOI: 10.48550/arXiv.1804.04637. arXiv: 1804.04637 [cs]. (Visited on 08/05/2025).

Aslan, Ömer and Abdullah Asim Yilmaz (2021). "A New Malware Classification Framework Based on Deep Learning Algorithms". In: *IEEE Access* 9, pp. 87936–87951. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3089586. (Visited on 07/28/2025).

Attaluri, Srilatha, Scott McGhee, and Mark Stamp (May 2009). "Profile Hidden Markov Models and Metamorphic Virus Detection". In: *Journal in Computer Virology* 5.2, pp. 151–169. ISSN: 1772-9904. DOI: 10.1007/s11416-008-0105-1. (Visited on 02/25/2025).

AV-TEST - The Independent IT-Security Institute (2025). *AV-ATLAS - Malware & PUA*. https://portal.av-atlas.org/malware. (Visited on 12/19/2024).

Bacci, Alessandro et al. (2018). "Impact of Code Obfuscation on Android Malware Detection Based onStatic and Dynamic Analysis". In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. Funchal, Madeira, Portugal: SCITEPRESS - Science and Technology Publications. DOI: 10.5220/0006642503790385. (Visited on 07/21/2025).

Baezner, Marie and Patrice Robin (Oct. 2017). *Stuxnet*. Report 4. ETH Zurich. DOI: 10.3929/ethz-b-000200661. (Visited on 07/22/2025).

Balakrishnan, Arini and Chloe Schulze (Dec. 2005). "Code Obfuscation Literature Survey". In.

Christodorescu, Mihai and Somesh Jha (Mar. 2004). "Static Analysis of Executables to Detect Malicious Patterns". In: 12.

Cochran, Kodi A. (2024). "The CIA Triad: Safeguarding Data in the Digital Realm". In: *Cybersecurity Essentials: Practical Tools for Today's Digital Defenders*. Ed. by Kodi A. Cochran. Berkeley, CA: Apress, pp. 17–32. ISBN: 979-8-8688-0432-8. DOI: 10.1007/979-8-8688-0432-8_2. (Visited on 07/22/2025).

Damodaran, Anusha et al. (Feb. 2017). "A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection". In: *Journal of Computer Virology and Hacking Techniques* 13.1, pp. 1–12. ISSN: 2263-8733. DOI: 10.1007/s11416-015-0261-z. (Visited on 07/29/2025).

Darem, Abdulbasit et al. (Dec. 2021). "Visualization and Deep-Learning-Based Malware Variant Detection Using OpCode-level Features". In: *Future Generation Computer Systems* 125, pp. 314–323. ISSN: 0167-739X. DOI: 10.1016/j.future.2021.06.032. (Visited on 07/28/2025).

Demırcı, Denız et al. (2022). "Static Malware Detection Using Stacked BiLSTM and GPT-2". In: *IEEE Access* 10, pp. 58488–58502. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3179384. (Visited on 02/25/2025).

Devlin, Jacob et al. (May 2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. (Visited on 02/28/2025).

European Union Agency for Cybersecurity (2024). *ENISA Threat Landscape 2024: July 2023 to June 2024*. LU: Publications Office. (Visited on 02/21/2025).

Habibi, Omar, Mohammed Chemmakha, and Mohamed Lazaar (Aug. 2023). "Performance Evaluation of CNN and Pre-trained Models for Malware Classification". In: *Arabian Journal for Science and Engineering* 48.8, pp. 10355–10369. ISSN: 2191-4281. DOI: 10.1007/s13369-023-07608-z. (Visited on 02/25/2025).

Idika, Mathur (Feb. 2007). "A Survey of Malware Detection Techniques". In.

Jain, M., W. Andreopoulos, and M. Stamp (2020). "Convolutional Neural Networks and Extreme Learning Machines for Malware Classification". In: *Journal of Computer Virology and Hacking Techniques* 16.3, pp. 229–244. DOI: 10.1007/s11416-020-00354-y.

Kakisim, Arzu Gorgulu, Sibel Gulmez, and Ibrahim Sogukpinar (Mar. 2022). "Sequential Opcode Embedding-Based Malware Detection Method". In: *Computers & Electrical Engineering* 98, p. 107703. ISSN: 0045-7906. DOI: 10.1016/j.compeleceng.2022.107703. (Visited on 07/28/2025).

Kamboj, Akshit et al. (Mar. 2023). "Detection of Malware in Downloaded Files Using Various Machine Learning Models". In: *Egyptian Informatics Journal* 24.1, pp. 81–94. ISSN: 1110-8665. DOI: 10.1016/j.eij.2022.12.002. (Visited on 07/28/2025).

Kaplan, Jared et al. (Jan. 2020). *Scaling Laws for Neural Language Models*. DOI: 10.48550/arXiv.2001.08361. arXiv: 2001.08361 [cs]. (Visited on 02/13/2025).

Karbab, ElMouatez Billah et al. (Mar. 2018). "MalDozer: Automatic Framework for Android Malware Detection Using Deep Learning". In: *Digital Investigation* 24, S48–S59. ISSN: 1742-2876. DOI: 10.1016/j.diin.2018.01.007. (Visited on 11/20/2024).

Khuri, Sami, Thomas Bäck, and Jörg Heitkötter (Apr. 1994). "The Zero/One Multiple Knapsack Problem and Genetic Algorithms". In: *Proceedings of the 1994 ACM Symposium on Applied Computing*. SAC '94. New York, NY, USA: Association for Computing Machinery, pp. 188–193. ISBN: 978-0-89791-647-9. DOI: 10.1145/326619.326694. (Visited on 08/04/2025).

Ki, Youngjoon, Eunjin Kim, and Huy Kang Kim (June 2015). "A Novel Approach to Detect Malware Based on API Call Sequence Analysis". In: *International Journal of Distributed Sensor Networks* 11.6, p. 659101. ISSN: 1550-1329. DOI: 10.1155/2015/659101. (Visited on 11/02/2024).

Kramer, Simon and Julian C. Bradfield (May 2010). "A General Definition of Malware". In: *Journal in Computer Virology* 6.2, pp. 105–114. ISSN: 1772-9904. DOI: 10.1007/s11416-009-0137-1. (Visited on 02/21/2025).

Li, Ce et al. (Nov. 2022). "DMalNet: Dynamic Malware Analysis Based on API Feature Engineering and Graph Learning". In: *Computers & Security* 122, p. 102872. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102872. (Visited on 11/27/2024).

Li, Yang et al. (Jan. 2023). "APIASO: A Novel API Call Obfuscation Technique Based on Address Space Obscurity". In: *Applied Sciences* 13.16, p. 9056. ISSN: 2076-3417. DOI: 10.3390/app13169056. (Visited on 02/25/2025).

M., Gopinath and Sibi Chakkaravarthy Sethuraman (Feb. 2023). "A Comprehensive Survey on Deep Learning Based Malware Detection Techniques". In: *Computer Science Review* 47, p. 100529. ISSN: 1574-0137. DOI: 10.1016/j.cosrev.2022.100529. (Visited on 11/20/2024).

Maniriho, Pascal, Abdun Naser Mahmood, and Mohammad Jabed Morshed Chowdhury (Mar. 2024). "A Systematic Literature Review on Windows Malware Detection: Techniques, Research Issues, and Future Directions". In: *Journal of Systems and Software* 209, p. 111921. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111921. (Visited on 02/18/2025).

Merabet, Hoda El and Abderrahmane Hajraoui (2019). "A Survey of Malware Detection Techniques Based on Machine Learning". In: *International Journal of Advanced Computer Science and Applications* 10.1. ISSN: 21565570, 2158107X. DOI: 10.14569/IJACSA.2019.0100148. (Visited on 11/20/2024).

National Audit Office (NAO) (Oct. 2017). *Investigation: WannaCry Cyber Attack and the NHS*. Tech. rep. London, England: National Audit Office.

Ni, Sang, Quan Qian, and Rui Zhang (Aug. 2018). "Malware Identification Using Visualization Images and Deep Learning". In: *Computers & Security* 77, pp. 871–885. ISSN: 0167-4048. DOI: 10.1016/j.cose.2018.04.005. (Visited on 02/21/2025).

Quiquet, François (Oct. 2023). *An analysis of the Viasat cyber attack with the MITRE ATT&CK® framework*. (Visited on 07/21/2025).

Radford, Alec et al. (n.d.). "Language Models Are Unsupervised Multitask Learners". In: ().

Rahali, Abir and Moulay A. Akhloufi (Oct. 2021). "MalBERT: Malware Detection Using Bidirectional Encoder Representations from Transformers". In: *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 3226–3231. DOI: 10.1109/SMC52423.2021.9659287. (Visited on 02/25/2025).

Rokon, Md Omar Faruk et al. (May 2020). *SourceFinder: Finding Malware Source-Code from Publicly Available Repositories*. DOI: 10.48550/arXiv.2005.14311. arXiv: 2005.14311 [cs]. (Visited on 07/31/2025).

Runwal, Neha, Richard M. Low, and Mark Stamp (May 2012). "Opcode Graph Similarity and Metamorphic Detection". In: *Journal in Computer Virology* 8.1, pp. 37–52. ISSN: 1772-9904. DOI: 10.1007/s11416-012-0160-5. (Visited on 02/25/2025).

Schultz, M.G. et al. (May 2001). "Data Mining Methods for Detection of New Malicious Executables". In: *Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001*, pp. 38–49. DOI: 10.1109/SECPRI.2001.924286. (Visited on 02/21/2025).

Sharma, Arindam, Pasquale Malacaria, and MHR Khouzani (June 2019). "Malware Detection Using 1-Dimensional Convolutional Neural Networks". In: *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 247–256. DOI: 10.1109/EuroSPW.2019.00034. (Visited on 07/28/2025).

Singh, Jagsir and Jaswinder Singh (May 2020). "Detection of Malicious Software by Analyzing the Behavioral Artifacts Using Machine Learning Algorithms". In: *Information and Software Technology* 121, p. 106273. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106273. (Visited on 07/28/2025).

Tan, Mingxing and Quoc V. Le (Sept. 2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. DOI: 10.48550/arXiv.1905.11946. arXiv: 1905.11946 [cs]. (Visited on 07/31/2025).

Tian, Ronghua et al. (2009). "An Automated Classification System Based on the Strings of Trojan and Virus Families". In: *2009 4th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 23–30. DOI: 10.1109/MALWARE.2009.5403021.

Vaswani, Ashish et al. (Aug. 2023). *Attention Is All You Need*. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs]. (Visited on 07/30/2025).

Wang, Haojun et al. (2021). "Deep Learning and Regularization Algorithms for Malicious Code Classification". In: *IEEE Access* 9, pp. 91512–91523. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3090464. (Visited on 07/28/2025).

Yang, Limin et al. (May 2021). "BODMAS: An Open Dataset for Learning Based Temporal Analysis of PE Malware". In: *2021 IEEE Security and Privacy Workshops (SPW)*. San Francisco, CA, USA: IEEE, pp. 78–84. ISBN: 978-1-6654-3732-5. DOI: 10.1109/SPW53761.2021.00020. (Visited on 08/05/2025).

You, Ilsun and Kangbin Yim (2010). "Malware Obfuscation Techniques: A Brief Survey". In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.

Appendix A

Implementation Details

A.1 Conversion of Executable Programs to Greyscale Images

Algorithm 1 Converts a program to an image

Require: Program $P \in [0, 255]^L$ of L bytes
Ensure: Matrix $M \in [0, 255]^{d \times d}$
1: $d \leftarrow \lceil \sqrt{L} \rceil$
2: Initialize $V \in [0, 255]^{d^2}$ with zeros
3: **for** $i \leftarrow 1$ to L **do**
4: $\quad V_i \leftarrow P_i$
5: **end for**
6: Reshape V into matrix $M \in [0, 255]^{d \times d}$ in row-major order
7: **return** M

A.2 Genetic Algorithm